Continuous Integration

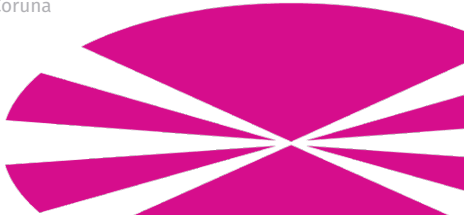
Ferramentas de Desenvolvemento

Javier Parapar López

Grao en Enxeñaría Informática · Facultade de Informática · Universidade da Coruña

Curso 2026/2027

GEI GRADO EN ENXEÑARÍA INFORMÁTICA



Where we are going

1. Continuous integration
2. Jenkins
3. A job, end to end
4. Reading the result
5. The pipeline as code
6. Working this way

SECTION 1

Continuous integration

1

Integration used to be a phase

For decades, teams wrote code separately for weeks and then spent a miserable fortnight putting it together. It had a name — *integration* — it was on the plan, and it always took longer than the plan said.

The definition

Continuous integration is *"a software development practice where each member of a team merges their changes into a codebase together with their colleagues' changes* ***at least daily****"*.

— Martin Fowler

Integration used to be a phase

For decades, teams wrote code separately for weeks and then spent a miserable fortnight putting it together. It had a name — *integration* — it was on the plan, and it always took longer than the plan said.

The definition

Continuous integration is *"a software development practice where each member of a team merges their changes into a codebase together with their colleagues' changes* ***at least daily****"*.

— Martin Fowler

Why it works

Conflicts do not grow linearly with time. Two days apart is an awkward merge; two months apart is a rewrite. Integrating constantly keeps every conflict small enough to be boring.

Integration used to be a phase

For decades, teams wrote code separately for weeks and then spent a miserable fortnight putting it together. It had a name — *integration* — it was on the plan, and it always took longer than the plan said.

The definition

Continuous integration is *"a software development practice where each member of a team merges their changes into a codebase together with their colleagues' changes* ***at least daily****"*.

— Martin Fowler

Why it works

Conflicts do not grow linearly with time. Two days apart is an awkward merge; two months apart is a rewrite. Integrating constantly keeps every conflict small enough to be boring.

Notice what the definition does **not** mention: a server, a tool, a plugin. CI is a **practice**. Jenkins is only what makes the practice cheap enough to keep.

Integration used to be a phase

For decades, teams wrote code separately for weeks and then spent a miserable fortnight putting it together. It had a name — *integration* — it was on the plan, and it always took longer than the plan said.

The definition

Continuous integration is *"a software development practice where each member of a team merges their changes into a codebase together with their colleagues' changes* ***at least daily****"*.

— Martin Fowler

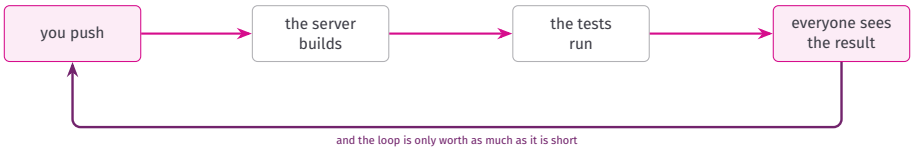
Why it works

Conflicts do not grow linearly with time. Two days apart is an awkward merge; two months apart is a rewrite. Integrating constantly keeps every conflict small enough to be boring.

Notice what the definition does **not** mention: a server, a tool, a plugin. CI is a **practice**. Jenkins is only what makes the practice cheap enough to keep.

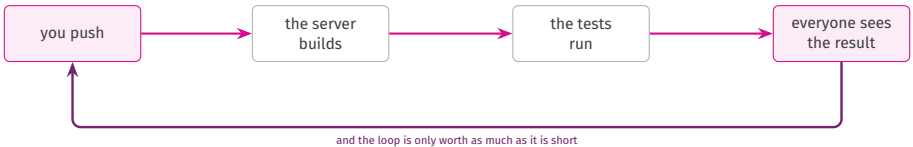
A team with a Jenkins server and week-long branches is not doing continuous integration. It owns a build server.

What is actually continuous is the feedback



The value of CI is not that a machine compiles your code. It is **how long you stay wrong**. A bug found four minutes after the push is a small correction while you still remember the change; found in three weeks, it is an investigation.

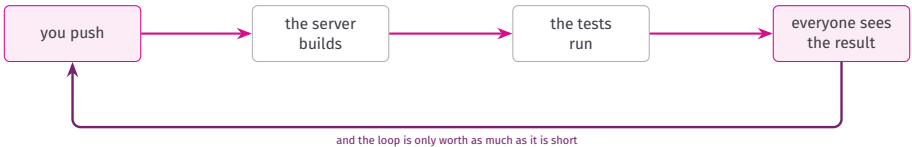
What is actually continuous is the feedback



The value of CI is not that a machine compiles your code. It is **how long you stay wrong**. A bug found four minutes after the push is a small correction while you still remember the change; found in three weeks, it is an investigation.

Hence the classic target, the ten-minute build: *"every minute chiseled off the build time is a minute saved for each developer every time they commit"*. Beyond about ten minutes people stop waiting, start batching, and the practice quietly dies.

What is actually continuous is the feedback



The value of CI is not that a machine compiles your code. It is **how long you stay wrong**. A bug found four minutes after the push is a small correction while you still remember the change; found in three weeks, it is an investigation.

Hence the classic target, the ten-minute build: *"every minute chiseled off the build time is a minute saved for each developer every time they commit"*. Beyond about ten minutes people stop waiting, start batching, and the practice quietly dies.

So build time is not a technical detail. It is the thing that decides whether anybody uses the pipeline at all.

When the build goes red

The rule is older than CI and comes from a factory floor: when the line stops, **stopping the line is the priority**. Kent Beck's version, which Fowler quotes: *"nobody has a higher priority task than fixing the build"*.

What to do

- **Say so**, in the team channel, before anyone else pulls it
- If the fix is obvious and small, **fix forward** — now, not after lunch
- If it is not, **revert** the change. Reverting is not a defeat; it is how the trunk stays usable for the other five people
- Then reproduce the failure locally and come back properly

When the build goes red

The rule is older than CI and comes from a factory floor: when the line stops, **stopping the line is the priority**. Kent Beck's version, which Fowler quotes: *"nobody has a higher priority task than fixing the build"*.

What to do

- **Say so**, in the team channel, before anyone else pulls it
- If the fix is obvious and small, **fix forward** — now, not after lunch
- If it is not, **revert** the change. Reverting is not a defeat; it is how the trunk stays usable for the other five people
- Then reproduce the failure locally and come back properly

And the worst case

- A flaky test — one that fails sometimes — is **worse than a failing one**. A failing test tells you something; a flaky test teaches the team to ignore red
- Once people re-run the build hoping for green, the pipeline has stopped being information
- So a flaky test is a **defect**: an issue in the tracker, fixed or quarantined, never re-run and forgotten

When the build goes red

The rule is older than CI and comes from a factory floor: when the line stops, **stopping the line is the priority**. Kent Beck's version, which Fowler quotes: *"nobody has a higher priority task than fixing the build"*.

What to do

- **Say so**, in the team channel, before anyone else pulls it
- If the fix is obvious and small, **fix forward** — now, not after lunch
- If it is not, **revert** the change. Reverting is not a defeat; it is how the trunk stays usable for the other five people
- Then reproduce the failure locally and come back properly

And the worst case

- A flaky test — one that fails sometimes — is **worse than a failing one**. A failing test tells you something; a flaky test teaches the team to ignore red
- Once people re-run the build hoping for green, the pipeline has stopped being information
- So a flaky test is a **defect**: an issue in the tracker, fixed or quarantined, never re-run and forgotten

The question that tells you how a team really works is not "does your build pass?" It is "how long was it red last time, and who noticed?"

What continuous integration asks of you

Almost none of it is about the CI server. It is about the four lessons you have already had:

THE PRACTICE	WHICH IS WHY
Everything in version control — code, tests, configuration, the build itself	the server can reconstruct your project from nothing *(Lesson 2)*
One command builds and tests it, from a terminal	the server has no IDE and nobody to click for it *(Lesson 4)*
Integrate in small pieces, often	every integration is small enough to understand *(Lesson 2)*
Tests run as part of the build, and failing tests fail it	green has to mean something *(Lesson 4)*
The result reaches the whole team, not just whoever pushed	a red build nobody sees is not a signal *(Lesson 3)*
It runs on a machine that resembles production	passing on your laptop was never the claim

What continuous integration asks of you

Almost none of it is about the CI server. It is about the four lessons you have already had:

THE PRACTICE	WHICH IS WHY
Everything in version control — code, tests, configuration, the build itself	the server can reconstruct your project from nothing *(Lesson 2)*
One command builds and tests it, from a terminal	the server has no IDE and nobody to click for it *(Lesson 4)*
Integrate in small pieces, often	every integration is small enough to understand *(Lesson 2)*
Tests run as part of the build, and failing tests fail it	green has to mean something *(Lesson 4)*
The result reaches the whole team, not just whoever pushed	a red build nobody sees is not a signal *(Lesson 3)*
It runs on a machine that resembles production	passing on your laptop was never the claim

Which is the honest summary of this lesson: if the previous four were done properly, continuous integration is mostly a configuration screen. If they were not, no server will save you.

SECTION 2

Jenkins

2

Jenkins

- An **automation server**, written in Java, free software under the MIT licence. It is the 2011 fork of Hudson, renamed after Oracle claimed the trademark
- Old, unfashionable and everywhere: two decades of use mean that whatever you need to connect to, **a plugin exists**
- Its model is small: a job says what to run and when, a build is one execution of it, and everything else — Git, Maven, SonarQube, Kubernetes — arrives as a plugin
- The controller schedules; agents do the work, each in a workspace of its own



current LTS **2.568.3** (September 2026)
needs Java 21 or newer

Jenkins

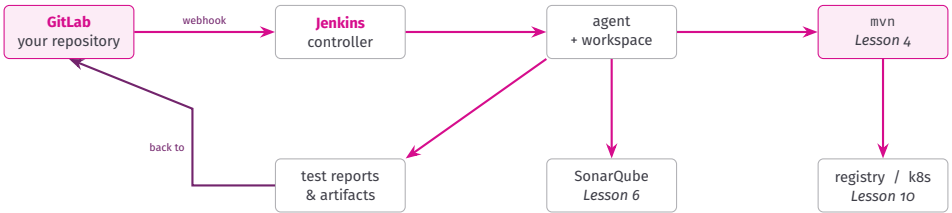
- An **automation server**, written in Java, free software under the MIT licence. It is the 2011 fork of Hudson, renamed after Oracle claimed the trademark
- Old, unfashionable and everywhere: two decades of use mean that whatever you need to connect to, **a plugin exists**
- Its model is small: a job says what to run and when, a build is one execution of it, and everything else — Git, Maven, SonarQube, Kubernetes — arrives as a plugin
- The controller schedules; agents do the work, each in a workspace of its own

The plugins are also the cost: they are what you update, what breaks, and what you have to trust. A CI server is a machine that runs arbitrary code with credentials — treat it like one.



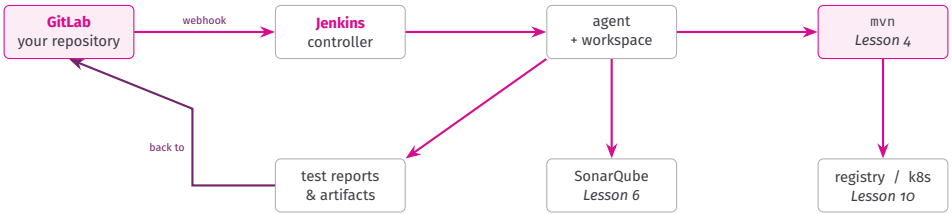
current LTS **2.568.3** (September 2026)
needs Java 21 or newer

Where the pieces sit



Read it once and the rest of the lesson is detail. GitLab tells Jenkins something happened; Jenkins gives an agent a clean workspace; the agent runs **your** build — the one from *Lesson 4*, unchanged — and then publishes what that build produced.

Where the pieces sit



Read it once and the rest of the lesson is detail. GitLab tells Jenkins something happened; Jenkins gives an agent a clean workspace; the agent runs **your** build — the one from *Lesson 4*, unchanged — and then publishes what that build produced.

Jenkins adds nothing to your build. It runs it somewhere else, on a schedule you did not have to remember, and tells people.

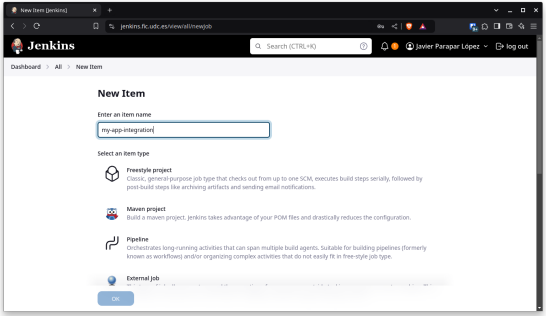
SECTION 3

A job, end to end

3

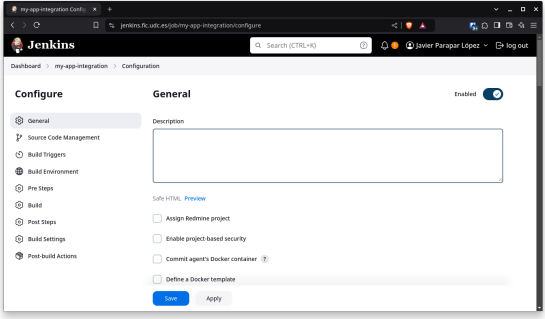
Creating a job

The first real decision: *Freestyle* does anything, *Maven* knows what a POM is, *Pipeline* reads its definition from your repository.



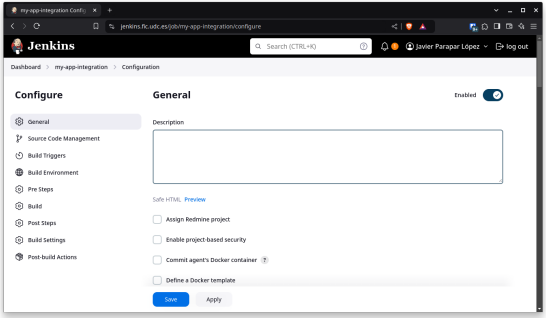
Creating a job

A name, the Redmine project of *Lesson 3*, and where the source comes from.



Creating a job

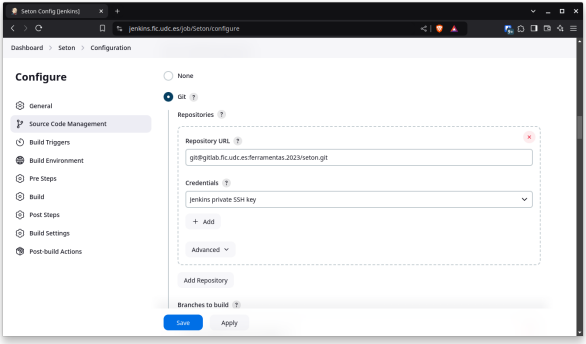
A name, the Redmine project of *Lesson 3*, and where the source comes from.



We use Maven: it reads your POM and reports modules and tests for free. Remember Pipeline — we return to it at the end.

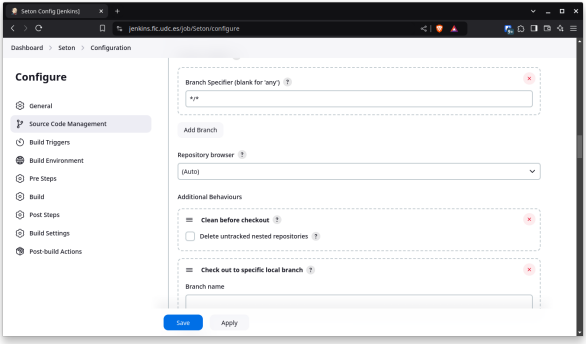
Where the code comes from

The repository, and the **credentials** Jenkins reads it with — the server has its own SSH key.



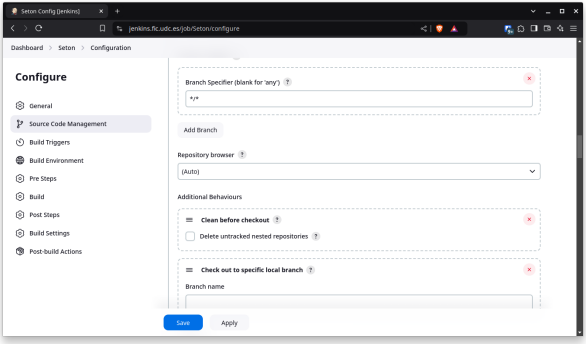
Where the code comes from

The branch, and the behaviour that matters: **clean before checkout**.



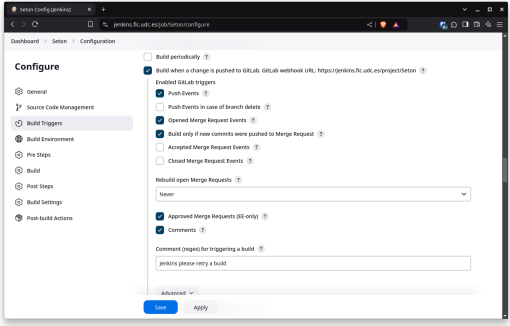
Where the code comes from

The branch, and the behaviour that matters: **clean before checkout**.



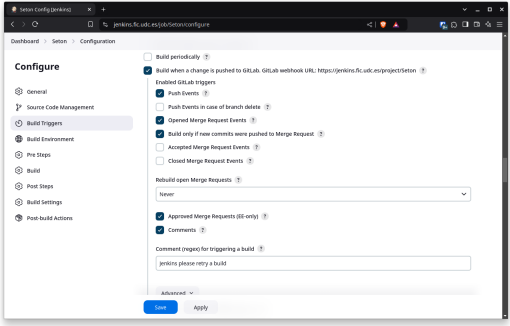
One checkbox holding the whole idea: a build that reuses state can pass for reasons you cannot reproduce.

When it runs: the merge request is the gate



Build triggers is where GitLab and Jenkins are wired together: a webhook on the GitLab project calls the Jenkins URL when something happens. Which *something* is a choice, and it is the important one on this screen.

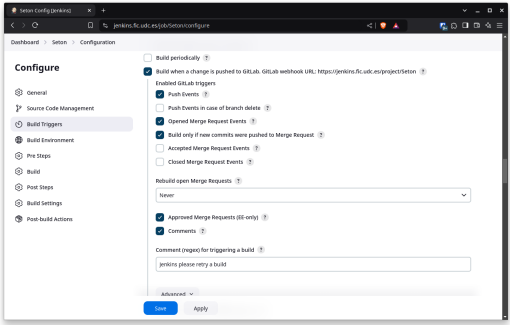
When it runs: the merge request is the gate



Build triggers is where GitLab and Jenkins are wired together: a webhook on the GitLab project calls the Jenkins URL when something happens. Which *something* is a choice, and it is the important one on this screen.

In this course the trigger that must work is **opened merge request events**. A branch still being written does not need the server's attention; a branch whose author says *"this is ready for* `main`*"* does.

When it runs: the merge request is the gate

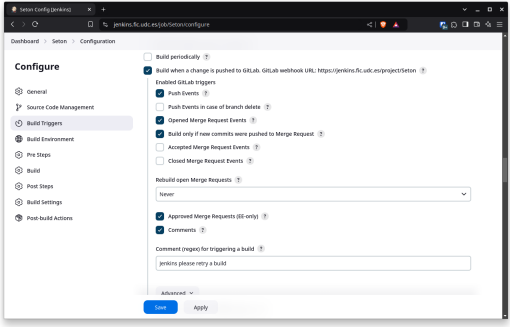


Build triggers is where GitLab and Jenkins are wired together: a webhook on the GitLab project calls the Jenkins URL when something happens. Which *something* is a choice, and it is the important one on this screen.

In this course the trigger that must work is **opened merge request events**. A branch still being written does not need the server's attention; a branch whose author says *"this is ready for* `main`*"* does.

That is the gate from *Lesson 2* — green pipeline plus one approval — and this checkbox is what makes it real.

When it runs: the merge request is the gate



Build triggers is where GitLab and Jenkins are wired together: a webhook on the GitLab project calls the Jenkins URL when something happens. Which *something* is a choice, and it is the important one on this screen.

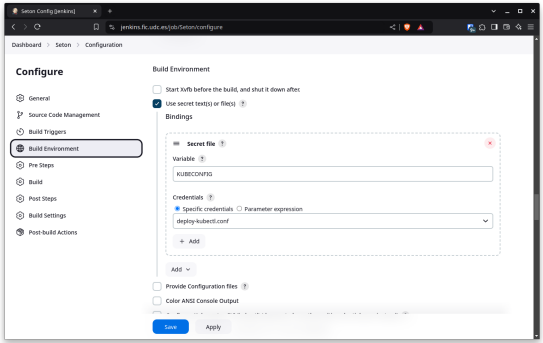
In this course the trigger that must work is **opened merge request events**. A branch still being written does not need the server's attention; a branch whose author says *"this is ready for* `main`*"* does.

That is the gate from *Lesson 2* — green pipeline plus one approval — and this checkbox is what makes it real.

Which is why your branches must stay short. The MR is where integration actually happens: a two-day branch means integrating every two days; a two-week branch makes the gate a formality at the end.

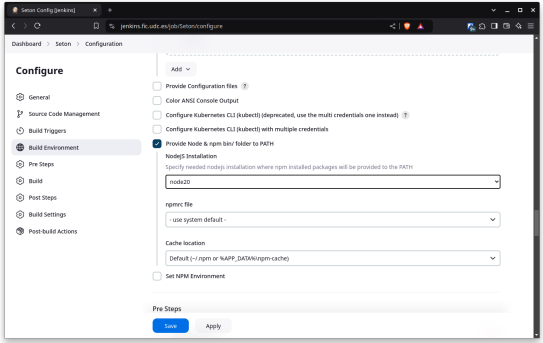
The environment, and the steps

Build environment: secrets are **injected** from the credentials store, never written down.



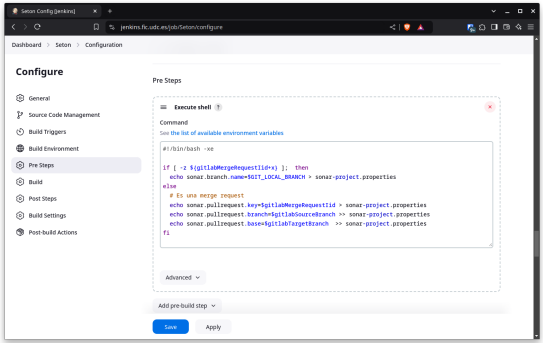
The environment, and the steps

The same section puts **Node and npm on the PATH** — half your project is a front end.



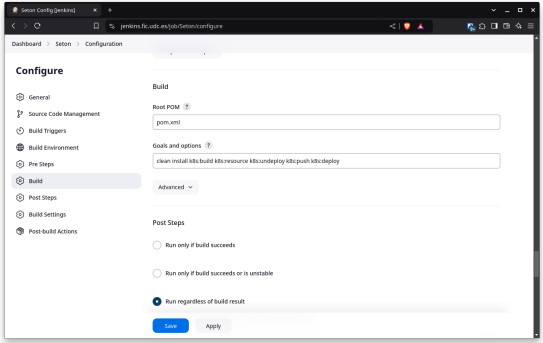
The environment, and the steps

Pre-steps: a script before the build. This one writes the Sonar properties.



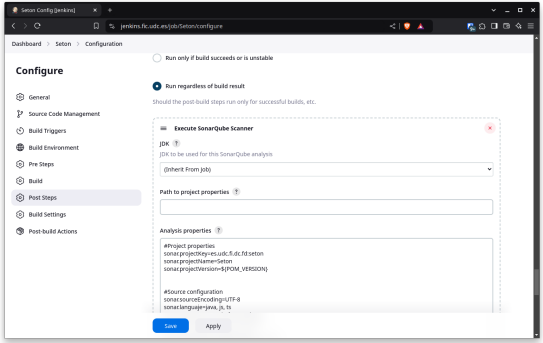
The environment, and the steps

Build: the root POM and the goals. This is where *Lesson 4* is executed.



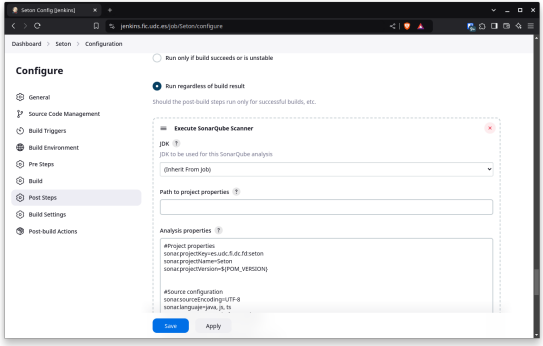
The environment, and the steps

Post-steps: afterwards, and whether it runs when the build failed.



The environment, and the steps

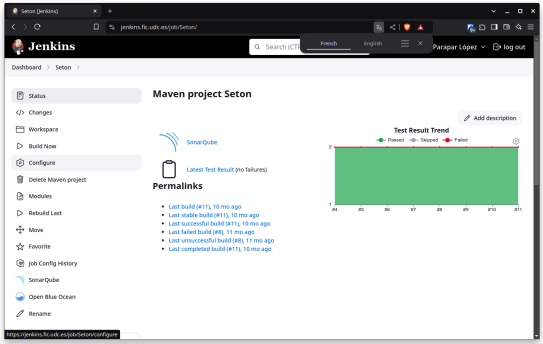
Post-steps: afterwards, and whether it runs when the build failed.



Stare at the fourth screen: the whole job is `mvn` and a list of goals. The rest is plumbing.

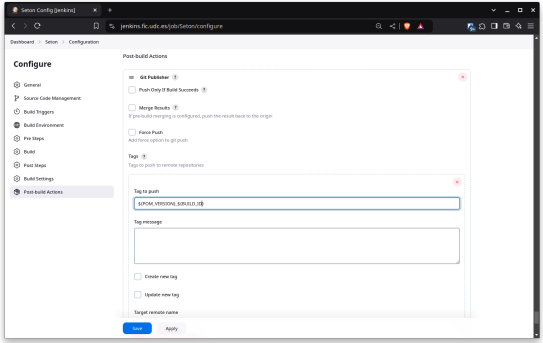
Changing a job, and what it leaves behind

A job is not written once — and every change is one more thing that lives only here.



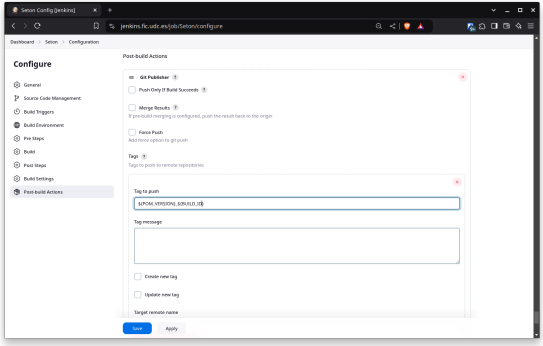
Changing a job, and what it leaves behind

Post-build actions: pushing a **tag** back to GitLab for the commit that was integrated.



Changing a job, and what it leaves behind

Post-build actions: pushing a **tag** back to GitLab for the commit that was integrated.



The traceability closes: issue, branch, merge request, build — and a tag on the exact commit that was validated.

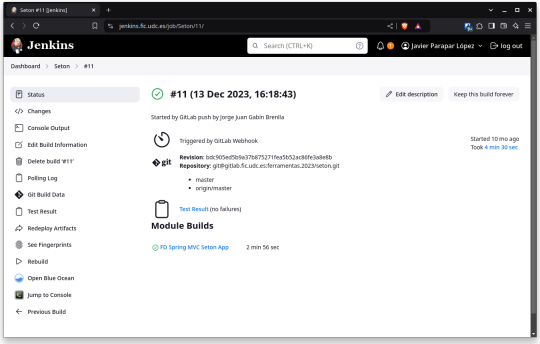
SECTION 4

Reading the result



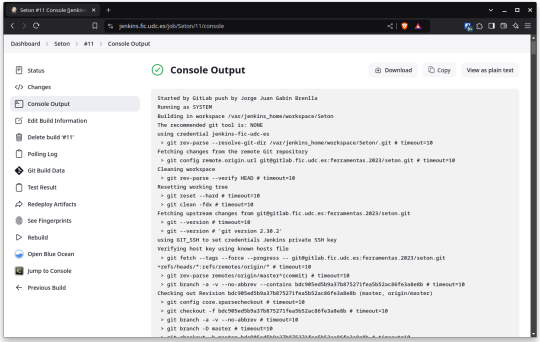
One build

Four questions at a glance: **who** triggered it, **which revision**, **how long**, and **did the tests pass**.



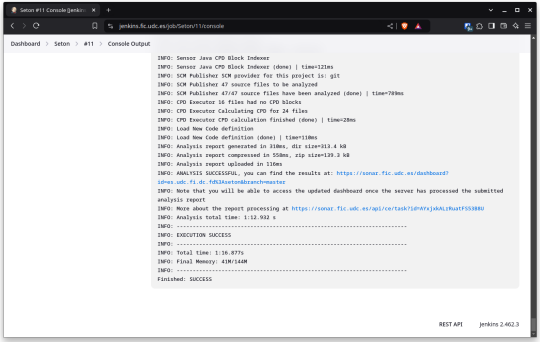
One build

The **console output** is the whole truth: every command, in order.



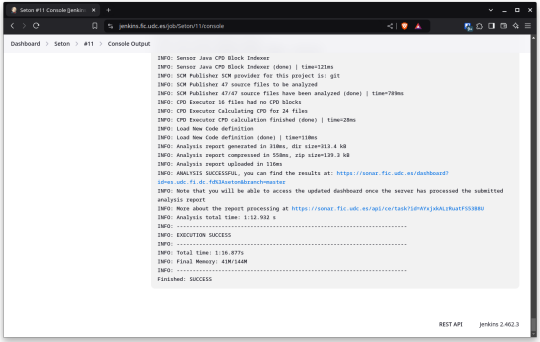
One build

And it ends with a verdict: `Finished:  SUCCESS`, or not.



One build

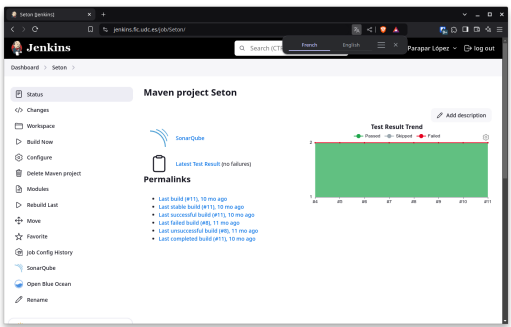
And it ends with a verdict: `Finished:  SUCCESS`, or not.



Read a failure from the first error, not the last: the last is usually Maven reporting that something above it broke.

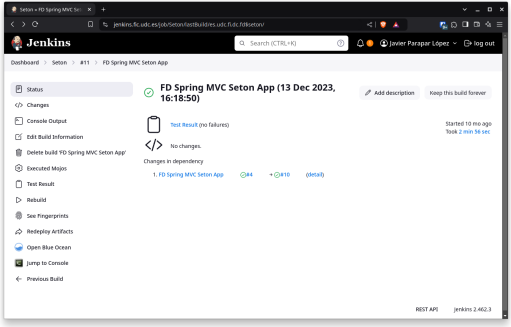
The reports are the point

The job page keeps the **history**: the test trend, and permalinks to the last stable, successful and failed build.



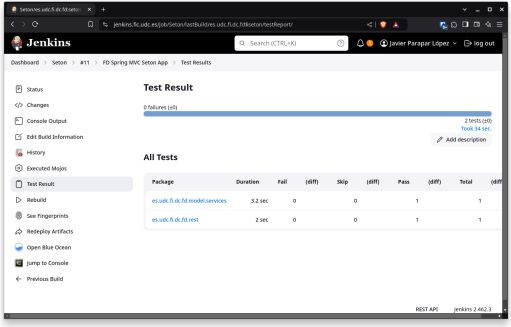
The reports are the point

Each build carries its own test result…



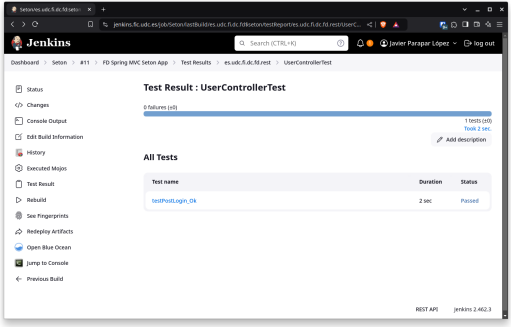
The reports are the point

… which opens into every package and class, with durations…



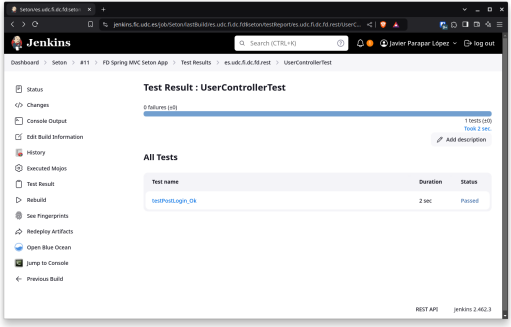
The reports are the point

… down to a single test method.



The reports are the point

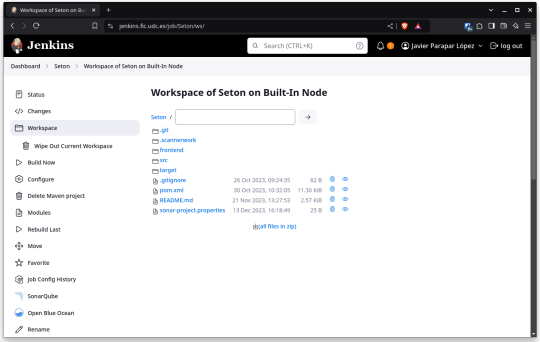
… down to a single test method.



None of it was configured: Maven wrote `surefire-reports` and Jenkins read it. The build produces, the server publishes.

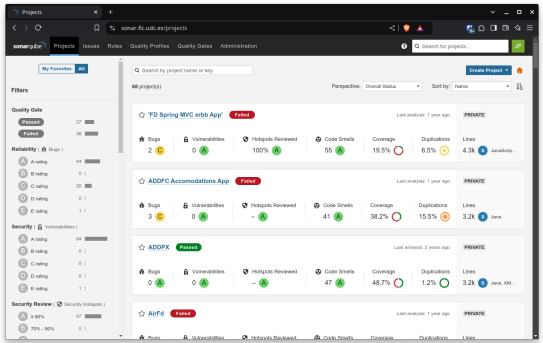
The workspace, and the hand-off

The **workspace**: the agent's copy of your project, exactly as the build left it.



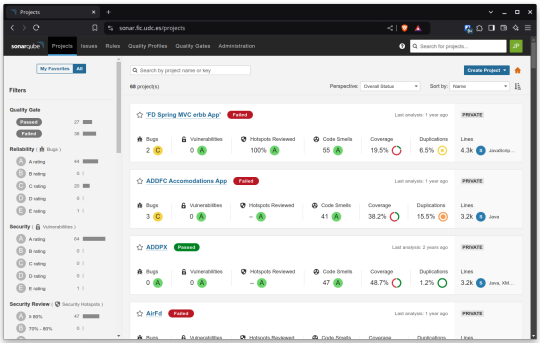
The workspace, and the hand-off

And what the build sent onwards — *Lesson 6* is already waiting in SonarQube.



The workspace, and the hand-off

And what the build sent onwards — *Lesson 6* is already waiting in SonarQube.



The workspace is a convenience, not a deliverable: what must survive is archived or published on purpose.

SECTION 5

The pipeline as code

5

One uncomfortable question

Everything this course has argued for says the same thing: what matters goes **in the repository**, where it is versioned, reviewed and recoverable. The code, the tests, the build, the documentation.

And yet

The job you have just seen lives **in the server**. It was configured by clicking. It is in no branch, no merge request reviewed it, and restoring the server from a backup puts it back to whatever it was then.

One uncomfortable question

Everything this course has argued for says the same thing: what matters goes **in the repository**, where it is versioned, reviewed and recoverable. The code, the tests, the build, the documentation.

And yet

The job you have just seen lives **in the server**. It was configured by clicking. It is in no branch, no merge request reviewed it, and restoring the server from a backup puts it back to whatever it was then.

The answer the industry reached

Put the pipeline in the repository too. A file — `Jenkinsfile`, `.gitlab-ci.yml`, `.github/workflows/` — **next to the code it builds**, changed in the same merge request as the change that needed it.

One uncomfortable question

Everything this course has argued for says the same thing: what matters goes **in the repository**, where it is versioned, reviewed and recoverable. The code, the tests, the build, the documentation.

And yet

The job you have just seen lives **in the server**. It was configured by clicking. It is in no branch, no merge request reviewed it, and restoring the server from a backup puts it back to whatever it was then.

The answer the industry reached

Put the pipeline in the repository too. A file — `Jenkinsfile`, `.gitlab-ci.yml`, `.github/workflows/` — **next to the code it builds**, changed in the same merge request as the change that needed it.

It also solves something the click-through job cannot: a branch can carry **its own** pipeline, reviewed with the rest of the change.

One uncomfortable question

Everything this course has argued for says the same thing: what matters goes **in the repository**, where it is versioned, reviewed and recoverable. The code, the tests, the build, the documentation.

And yet

The job you have just seen lives **in the server**. It was configured by clicking. It is in no branch, no merge request reviewed it, and restoring the server from a backup puts it back to whatever it was then.

The answer the industry reached

Put the pipeline in the repository too. A file — `Jenkinsfile`, `.gitlab-ci.yml`, `.github/workflows/` — **next to the code it builds**, changed in the same merge request as the change that needed it.

It also solves something the click-through job cannot: a branch can carry **its own** pipeline, reviewed with the rest of the change.

We keep the configured job here because it is simpler to teach and the screens are in front of you — but you should leave knowing it is the old way, and why.

The same job, as a `Jenkinsfile`

```
pipeline {
    agent any
    stages {
        stage('Build')  { steps { sh 'mvn -B clean verify' } }
        stage('Sonar')  { steps { sh 'mvn -B sonar:sonar' } }
    }
    post {
        always  { junit 'target/surefire-reports/*.xml'
                  archiveArtifacts artifacts: 'target/*.jar' }
        failure { echo 'Tell the team. Do not walk away.' }
    }
}
```

- `agent` says **where** it runs, `stages` are the named steps you see in the progress bar, `post` is what happens afterwards — including when the build failed
- The file sits in the root of the repository. A **multibranch** Jenkins job finds it by itself, in every branch that has one
- And it is the same `mvn` command you run locally. Anything else would be a second, untested definition of your build

The same job, as a `Jenkinsfile`

```
pipeline {
    agent any
    stages {
        stage('Build')  { steps { sh 'mvn -B clean verify' } }
        stage('Sonar')  { steps { sh 'mvn -B sonar:sonar' } }
    }
    post {
        always  { junit 'target/surefire-reports/*.xml'
                  archiveArtifacts artifacts: 'target/*.jar' }
        failure { echo 'Tell the team. Do not walk away.' }
    }
}
```

- `agent` says **where** it runs, `stages` are the named steps you see in the progress bar, `post` is what happens afterwards — including when the build failed
- The file sits in the root of the repository. A **multibranch** Jenkins job finds it by itself, in every branch that has one
- And it is the same `mvn` command you run locally. Anything else would be a second, untested definition of your build

Ten lines replace nine configuration screens — and, unlike the screens, a merge request can review them.

The same idea, where your code already lives

.gitlab-ci.yml

```yaml
stages: [build, test]

maven:
  stage: build
  image: maven:3.9-eclipse-temurin-25
  script:
    - mvn -B clean verify
  artifacts:
    reports:
      junit: target/surefire-reports/*.xml
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```

- GitLab has CI built in: no separate server, and the pipeline appears **inside the merge request** you already opened
- A runner is what Jenkins calls an agent; `image` is the container it runs in — *Lesson 10*
- That last `rules` block is the gate again: run **only for merge requests**

JENKINS	GITLAB CI
job	pipeline
stage	stage
agent	runner
`Jenkinsfile`	`.gitlab-ci.yml`
plugin	template / image

The same idea, where your code already lives

.gitlab-ci.yml

```yaml
stages: [build, test]

maven:
  stage: build
  image: maven:3.9-eclipse-temurin-25
  script:
    - mvn -B clean verify
  artifacts:
    reports:
      junit: target/surefire-reports/*.xml
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```

- GitLab has CI built in: no separate server, and the pipeline appears **inside the merge request** you already opened
- A runner is what Jenkins calls an agent; `image` is the container it runs in — *Lesson 10*
- That last `rules` block is the gate again: run **only for merge requests**

JENKINS	GITLAB CI
job	pipeline
stage	stage
agent	runner
`Jenkinsfile`	`.gitlab-ci.yml`
plugin	template / image

Different words, same three ideas: something happened, run this somewhere clean, tell everyone. If you understand the Jenkins job, you can read this file on your first day at work.

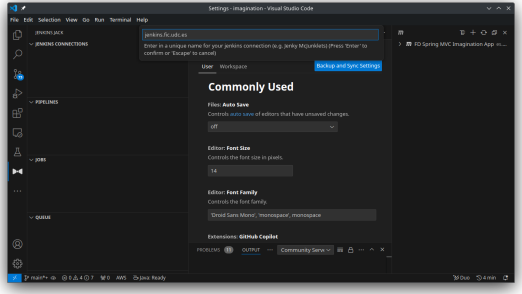
SECTION 6

Working this way

6

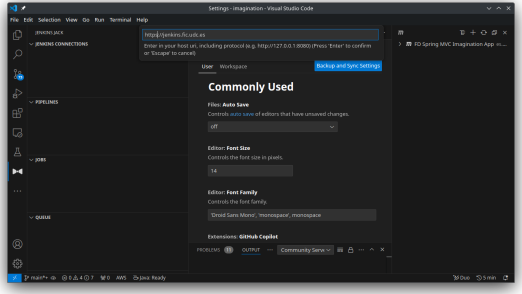
Jenkins from the editor

The **Jenkins Jack** extension connects VS Code to a Jenkins server.



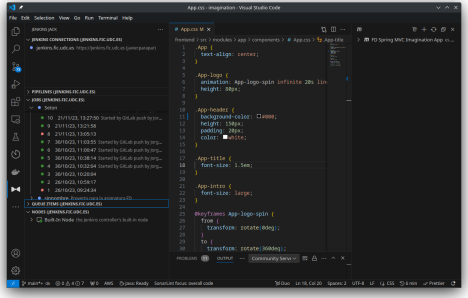
Jenkins from the editor

It needs the host and credentials — your own, not the team's.



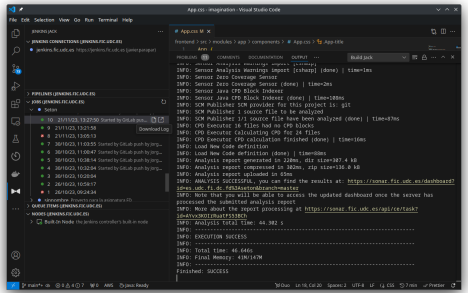
Jenkins from the editor

Jobs and builds then appear in the sidebar, green and red.



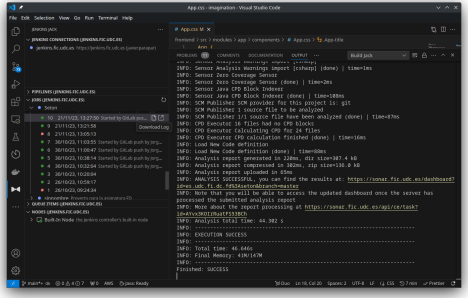
Jenkins from the editor

And the log comes to the editor, without a browser.



Jenkins from the editor

And the log comes to the editor, without a browser.



Same principle as Lesson 3: information must appear where the decision is taken. A red build in a closed tab is not feedback.

The rules, for the assignment

When it runs

- The pipeline runs on **merge requests**, not on every push to every branch. Opening the MR is what says *“this is ready for `main`”*
- So **open the MR early** and keep branches short — that is what keeps integration frequent, not the trigger configuration
- `main` is also built, and it must be **green**. If it is not, that is the team’s most urgent task

What it means

- **No merge on red.** Not "it’s unrelated", not "it’s the server". Red means the MR waits
- **Nobody disables a test to go green.** We read the history, and a deleted test is very visible
- A **flaky** test is an issue in Redmine, not a re-run habit
- The build result is **visible to the whole team**, and whoever broke it says so

The rules, for the assignment

When it runs

- The pipeline runs on **merge requests**, not on every push to every branch. Opening the MR is what says *"this is ready for `main`"*
- So **open the MR early** and keep branches short — that is what keeps integration frequent, not the trigger configuration
- `main` is also built, and it must be **green**. If it is not, that is the team's most urgent task

What it means

- **No merge on red.** Not "it's unrelated", not "it's the server". Red means the MR waits
- **Nobody disables a test to go green.** We read the history, and a deleted test is very visible
- A **flaky** test is an issue in Redmine, not a re-run habit
- The build result is **visible to the whole team**, and whoever broke it says so

All of it is in the record: which MRs had a pipeline, which were merged red, how long `main` stayed broken, and whether the number of tests ever went down. That history is what we read when marking.

Your turn

1. Create your team's job in `jenkins.fic.udc.es` and connect it to your GitLab project, with the webhook firing on **merge request events**.
2. Open a merge request from a branch and watch the build start on its own. Nobody presses anything.
3. **Break it on purpose**: push a commit with a failing test, see the build go red, and find the failing method from the Jenkins report in under a minute.
4. Time your build. If it is over ten minutes, that is an issue for the iteration, not a fact of life.
5. Agree, in writing, who watches the pipeline and what the team does when `main` goes red.
6. Write the equivalent `Jenkinsfile` for your job — you do not have to use it, but you should be able to read it.

Your turn

1. Create your team's job in `jenkins.fic.udc.es` and connect it to your GitLab project, with the webhook firing on **merge request events**.
2. Open a merge request from a branch and watch the build start on its own. Nobody presses anything.
3. **Break it on purpose**: push a commit with a failing test, see the build go red, and find the failing method from the Jenkins report in under a minute.
4. Time your build. If it is over ten minutes, that is an issue for the iteration, not a fact of life.
5. Agree, in writing, who watches the pipeline and what the team does when `main` goes red.
6. Write the equivalent `Jenkinsfile` for your job — you do not have to use it, but you should be able to read it.

The third is the one that teaches the most, and the one teams skip. A pipeline you have never seen fail is a pipeline you do not know how to use.

What to take away

CI is a practice, not a server. Integrate at least daily, in small pieces; the server only makes it cheap.

What is continuous is the feedback. Ten minutes, not an hour — past that, people stop waiting and the practice dies.

A red build is the team's most urgent task, and a flaky test is worse than a failing one.

The server publishes what your build produced. It adds nothing. If it is not in the build, the pipeline cannot report it.

The pipeline belongs in the repository too, for exactly the reasons everything else does.

What to take away

CI is a practice, not a server. Integrate at least daily, in small pieces; the server only makes it cheap.

What is continuous is the feedback. Ten minutes, not an hour — past that, people stop waiting and the practice dies.

A red build is the team's most urgent task, and a flaky test is worse than a failing one.

The server publishes what your build produced. It adds nothing. If it is not in the build, the pipeline cannot report it.

The pipeline belongs in the repository too, for exactly the reasons everything else does.

Next: *Lesson 6* — the other thing the server can tell you on every merge request, which is not whether the code works but whether it is any good.

Appendix: declarative pipeline, a little further

```
pipeline {
  agent any
  options     { timeout(time: 20, unit: 'MINUTES'); disableConcurrentBuilds() }
  environment { SONAR_HOST = 'https://sonar.fic.udc.es' }
  stages {
    stage('Build')    { steps { sh 'mvn -B clean verify' } }
    stage('Frontend') { steps { sh 'npm ci && npm run build' } }
    stage('Sonar')    {
      when  { branch 'main' }                    // only where it is worth it
      steps { withCredentials([string(credentialsId: 'sonar-token',
                                      variable: 'TOKEN')]) {
                sh 'mvn -B sonar:sonar -Dsonar.token=$TOKEN' } } }
  }
  post { always { junit 'target/surefire-reports/*.xml' } }
}
```

`options` bounds the build (a pipeline with no timeout will eventually hang forever) · `when` makes a stage conditional · `withCredentials` is how a secret enters, for the length of one block and masked in the log · `parallel` runs stages at the same time, which is the first thing to try when ten minutes becomes twenty.

Appendix: reading a red build

WHAT YOU SEE	WHERE TO LOOK FIRST
Compilation error	The first error in the console, not the last. Usually a file that was never committed
Tests fail on the server, pass locally	Order, time zone, locale, a leftover database, or a file path that only exists on your machine
Fails only sometimes	A flaky test. File it as an issue; do not re-run until it is green
`Could not resolve dependencies`	A version that is not in the repository, or a `jar` that only ever existed in somebody's `.m2` *(Lesson 4)*
Hangs and times out	A build waiting for an answer nobody is there to give
Green, but nothing was actually run	Check the goals. `-DskipTests` has a way of surviving a debugging session
Worked yesterday, nothing changed	Something moved: a floating version, a `-SNAPSHOT`, or a lockfile that is not committed

The list is short because the causes are few. Almost every red build is one of these seven, and five of them are the build's fault rather than the code's.